

Symmetry-Preserving Architecture for Multi-NUMA Environments (SPANE): A Deep Reinforcement Learning Approach for Dynamic VM Scheduling

Chan Tin Ping, Yunlong Cheng, Yizhan Zhu, Xiaofeng Gao*, Guihai Chen

Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China.

{chantp, aweft, yz518348, gaoxiaofeng, chen-gh}@sjtu.edu.cn

Abstract—As cloud computing continues to evolve, the adoption of multi-NUMA (Non-Uniform Memory Access) architecture by cloud service providers has introduced new challenges in virtual machine (VM) scheduling. To address these challenges and more accurately reflect the complexities faced by modern cloud environments, we introduce the Dynamic VM Allocation problem in Multi-NUMA PM (DVAMP). We formally define both offline and online versions of DVAMP as mixed-integer linear programming problems, providing a rigorous mathematical foundation for analysis. A tight performance bound for greedy online algorithms is derived, offering insights into the worst-case optimality gap as a function of the number of physical machines and VM lifetime variability. To address the challenges posed by DVAMP, we propose SPANE (Symmetry-Preserving Architecture for Multi-NUMA Environments), a novel deep reinforcement learning approach that exploits the problem's inherent symmetries. SPANE produces invariant results under arbitrary permutations of physical machine states, enhancing learning efficiency and solution quality. Extensive experiments conducted on the Huawei-East-1 dataset demonstrate that SPANE outperforms existing baselines, reducing average VM wait time by 45%. Our work contributes to the field of cloud resource management by providing both theoretical insights and practical solutions for VM scheduling in multi-NUMA environments, addressing a critical gap in the literature and offering improved performance for real-world cloud systems.

Index Terms—Virtual Machine Scheduling, Multi-NUMA, Symmetry, Deep Reinforcement Learning

I. INTRODUCTION

Cloud computing has revolutionized the way computational resources are delivered and consumed, offering unprecedented flexibility, scalability, and cost-effectiveness. Virtual machines (VM), the fundamental computing unit to the cloud services, enable cloud service providers (CSPs) to deliver scalable and flexible computing resources on demand. VMs are software implementations of physical machines (PM), each capable of hosting different applications and operating systems. Effectively scheduling VMs to PMs becomes a more and more crucial problem in the cloud due to the dynamic behavior of VMs, the growing number of requests and the need to guarantee Quality of Service (QoS) to users.

*Corresponding author: Xiaofeng Gao (gaoxiaofeng@sjtu.edu.cn).

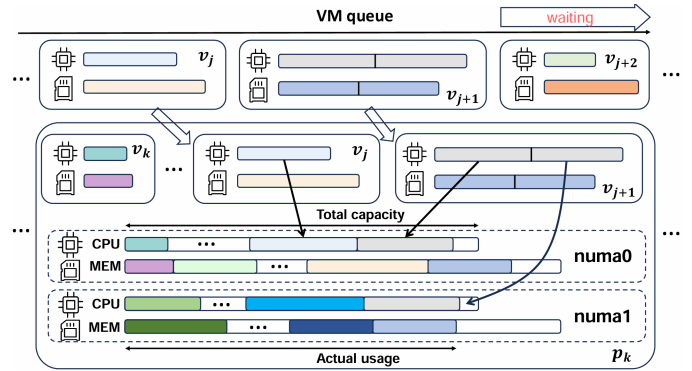


Fig. 1. Illustration of Multi-NUMA VM scheduling. Each PM consists of two NUMA nodes, with VMs deployed in one of three scenarios: (1) allocated entirely to a single NUMA node (e.g., v_j on NUMA 0), (2) evenly distributed across both NUMA nodes within the same PM (e.g., v_{j+1}), or (3) delayed due to insufficient resources (e.g., v_{j+2}).

Traditionally, the VM scheduling problem is modeled as a multi-dimensional vector bin-packing problem [1], [2]. Given a set of VMs with varying sizes, the goal is to minimize the number of PMs needed to accommodate these VMs without exceeding capacity limits. VMs are treated as items and PMs as bins, with constraints on multiple resources such as CPU, memory, disk, and network. Many classical optimization methods, including greedy and online algorithms, have been proposed to solve this problem [2]–[5].

Multi-Non-Uniform Memory Access (NUMA) architecture, widely adopted in modern servers, improves system performance by reducing memory access latency through closer proximity of memory to processors [6]. Many servers, such as AWS EC2 c5.24xlarge [7], feature two NUMA nodes. Compared to servers without NUMA, it offers more placement options, such as allocating VMs to a single NUMA node or splitting them across nodes, increasing the combinatorial complexity of scheduling. Fig. 1 illustrates the deployment of small and large VMs to PMs with two NUMA nodes: small VMs can be allocated to a single NUMA node, while large VMs may span multiple nodes depending on resource availability [8], [9]. When PMs lack sufficient capacity, VM

requests must wait for resources, creating a variant of the bin-packing problem. Unlike the static nature of bin-packing, VM requests in real-world cloud environments arrive sequentially and have finite lifespans, making the problem dynamic and challenging to solve.

To better capture the challenges faced by CSPs, we define the **Dynamic VM Allocation problem in Multi-NUMA PM (DVAMP)** and present both offline and online variants. In DVAMP, the number of PMs is fixed, and VM requests must be allocated to a single NUMA node or distributed across both NUMA nodes. When immediate deployment is impossible due to resource constraints, VM requests are queued until resources become available. The primary goal of DVAMP is to minimize total VM wait time. We also provide a performance bound for online algorithms, establishing a lower bound for the worst-case optimality gap between them and the optimal.

Traditional heuristics, while commonly used, typically operate within a predefined set of rules, limiting their ability to generalize across varying cloud conditions and workloads. Moreover, in dynamic environments like cloud computing, optimal decisions are not immediately obvious. Deep Reinforcement Learning (DRL), by contrast, can continuously adapt to these changing conditions by learning from past experiences and receiving real-time feedback. This adaptability allows DRL to effectively handle the dynamic nature of VM scheduling [10]. However, the large state and action spaces in cloud environments make both training and inference computationally expensive [11].

In response to these challenges, we propose **SPANE**, a **Symmetry-Preserving Architecture** designed specifically for Multi-NUMA Environments. SPANE exploits the intrinsic symmetry of the DVAMP problem, where the ordering of hosts in the state space does not affect the scheduling outcome. By leveraging this symmetry, SPANE improves action prediction efficiency and accuracy at each scheduling step.

Extensive experiments using real-world cloud traces validate the effectiveness of SPANE, which reduces average VM wait time by 45% compared to state-of-the-art methods. SPANE also demonstrates significant flexibility, applying effectively to VM scheduling scenarios with varying numbers of PMs. This ensures its applicability to practical cloud environments with highly dynamic and complex resource demands.

Contributions. We make the following contributions:

- 1) A formal definition of the DVAMP problem, including both offline and online versions. The offline version is formulated as a mixed-integer linear programming problem (MILP), establishing a rigorous mathematical foundation for detailed analysis and effective solution development. Section II provides the full details of this formulation.
- 2) A derivation of a tight performance bound for greedy online algorithms in the DVAMP problem. This analysis shows the worst-case optimality gap is approximately $\frac{m-1}{2m-1}(\mu - 1)$ times the total time-resource of VM requests, where

m is the number of physical machines and μ represents VM lifetime variability. This result offers insights into the limitations of online scheduling strategies and clarifies the impact of VM lifetime variability on system performance. Details are provided in Section III.

- 3) The development of SPANE, a novel deep neural network architecture that exploits the inherent symmetries of the DVAMP problem. SPANE produces invariant results under arbitrary permutations of PM states, enhancing learning efficiency and solution quality. The architecture and its implementation are discussed in Section IV.
- 4) The practical effectiveness of the proposed approach is demonstrated through extensive experiments on the Huawei-East-1 dataset [12]. The experimental setup, results, and analysis are detailed in Section V. Code and data are publicly available.¹

II. PROBLEM STATEMENT AND MATH FORMULATION

This section presents a formal formulation of DVAMP problem, encompassing both offline and online variants. We first introduce the system model, followed by detailed formulations of the offline and online problems.

A. System Model

Definition 1 (VM Requests). Let $V = \{v_1, v_2, \dots, v_n\}$ denote the set of VM requests, where v_j represents the j -th request. Each VM request is characterized by three essential attributes:

- 1) Arrival time at_j : The time at which v_j is received by the system. Notice that VMs are processed in sequential order, which means $at_j < at_{j+1}$ for all j .
- 2) Active time length lt_j : The duration for which v_j needs to be operational. The term "dynamic" in the problem nomenclature refers to the termination of VMs during the scheduling process.
- 3) Resource requirements r_j^d : The amount of each resource type d required by the VM, where $d \in [D]$ and D represents the total number of resource types (e.g., CPU cores, memory).

Definition 2 (Server Infrastructure). The server infrastructure consists of m PMs, each equipped with two NUMA nodes. Each NUMA node has the same resource capacity limit R_d for each resource type $d \in [D]$. The total resource utilization of all VMs on a NUMA node must not exceed its capacity limit R_d for any resource type. VM deployment adheres to one of two strategies:

- 1) Allocation to a single NUMA node within the same PM: The chosen node's resource utilization increases by the full resource requirement of the VM.
- 2) Equitable distribution across both NUMA nodes within the same PM: Both NUMA nodes experience an increase in resource utilization equal to half of the VM's resource needs.

¹<https://github.com/ChanTinPing/SPANE>

TABLE I
SYMBOLS AND THEIR MEANINGS

Symbols	Meaning
$[i]$	$\{1, 2, \dots, i\}$
D	Dimension of resources
V	Total VM request set
v_j	The j -th VM request
n	Length of total VM request set V
r_j^d	d -th resource requirement of v_j
at_j	Arrival time of request v_j
lt_j	Length of the time duration when v_j active
div_j	Function that determines whether v_j needs to be divided; output $\in \{0, 1\}$
p_k	k -th PM
m	Number of PMs
$u_{k,i}^d(t)$	The utilization of the d -th resource in the i -th NUMA of p_k at time t
R_d	The maximum capacity of d -th resource of all NUMA nodes
$x_{k,i,j}$	Whether i -th NUMA of p_k hosts v_j
$\delta_j(t)$	Whether v_j is active at time t

The deployment strategy for v_j is determined by the variable div_j , defined as:

$$div_j = \begin{cases} 1, & \text{if } r_j^{d_{div}} \geq c_{div} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

where 1 indicates equitable distribution across both NUMA nodes. This approach balances performance and resource utilization: smaller VMs benefit from single-node allocation, boosting performance, while distributing larger VMs across nodes enables the cluster to host more VMs overall.

B. Offline DVAMP Problem

Definition 3 (Offline DVAMP Problem). Given a set of VM requests $V = \{v_1, v_2, \dots, v_n\}$, number of PMs m , and maximum resource capacity R_d for each resource type $d \in [D]$, the offline DVAMP Problem aims to:

- 1) Determine the optimal NUMA node(s) deployment for each VM request.
- 2) Ensure that resource utilization of all NUMA nodes does not exceed R_d at any time.
- 3) Minimize the total wait time across all VM deployments.

The problem is subject to potential deployment delays.

Remark 1. In this context, "wait time" refers to the interval between a VM's arrival time and its start time, occurring when deployment is postponed (queuing time).

The complete mathematical formulation of the offline DVAMP problem is presented in (2). Table I provides a comprehensive list of symbols and their corresponding definitions used in this formulation.

In the DVAMP problem, the VM wait time is quantified as the difference between its start and arrival time. The objective function in (2a) aims to minimize the total wait time across all VMs. The output of the problem, as indicated in (2b), consists

of the deployment decision variable $x_{k,i,j}$ and the start time st_j for each VM. Here, $x_{k,i,j} = 1$ if v_j is deployed on the i -th NUMA node of p_k , and $x_{k,i,j} = 0$ otherwise.

Minimize:

$$WT = \sum_{j=1}^n (st_j - at_j) \quad (2a)$$

Output:

$$\forall k \in [m], \forall i \in \{0, 1\}, \forall j \in [n] :$$

$$x_{k,i,j}, st_j \quad (2b)$$

Constraints:

$$\forall k \in [m], \forall i \in \{0, 1\}, \forall d \in [D], \forall t > 0 :$$

$$u_{k,i}^d(t) \leq R_d \quad (2c)$$

$$u_{k,i}^d(t) = \sum_{j=1}^n \delta_j(t) x_{k,i,j} \gamma_j r_j^d \quad (2d)$$

$$\gamma_j = 1 - \frac{div_j}{2} \quad (2e)$$

$$\delta_j(t) = \begin{cases} 1, & st_j \leq t < st_j + lt_j \\ 0, & \text{otherwise} \end{cases} \quad (2f)$$

$$\forall k \in [m], \forall i \in \{0, 1\}, \forall j \in [n] :$$

$$x_{k,i,j} \in \{0, 1\} \quad (2g)$$

$$x_{k,0,j} = x_{k,1,j} \quad \text{if } div_j = 1 \quad (2h)$$

$$\forall j \in [n] :$$

$$\sum_{k=1}^m \sum_{i=0}^1 x_{k,i,j} = 1 + div_j \quad (2i)$$

$$\forall j \in [n] :$$

$$st_{j-1} < st_j \quad (2j)$$

$$at_j \leq st_j \quad (2k)$$

NUMA Resource Constraints: The core constraint (2c) ensures that at any given moment, the resource utilization of all NUMA nodes does not exceed the maximum capacity R_d . Equations (2d) through (2f) delineate the calculation of the resource utilization $u_{k,i}^d(t)$. The variable $\delta_j(t)$ indicates whether v_j is active at time t , as defined in (2f). Consequently, $\delta_j(t) x_{k,i,j}$ determines whether v_j is active in the i -th NUMA node of p_k at time t . For each active v_j , its contribution to the resource utilization $u_{k,i}^d(t)$ is determined by $\gamma_j r_j^d$, where γ_j is defined in (2e). This coefficient accounts for the deployment strategy: $\gamma_j = 1$ if $div_j = 0$ (single NUMA deployment), and $\gamma_j = 0.5$ if $div_j = 1$ (equitable distribution across both NUMA nodes). Although these four equations are presented separately to enhance clarity, they can be consolidated into a single equation.

Deployment Decision Constraints: Constraints (2g)–(2i) govern the deployment decision variable $x_{k,i,j}$. (2g) defines $x_{k,i,j}$ as a binary variable. (2h) ensures consistency for VMs that are evenly distributed across two NUMA nodes. (2i)

guarantees that each VM is deployed to either one NUMA node or two NUMA nodes, depending on the value of div_j .

Timing Constraints: Finally, constraints (2j)–(2k) enforce the proper ordering and timing of VM deployments, ensuring that VMs are processed sequentially and that no VM starts before its arrival time.

C. Online DVAMP Problem

In reality, cloud servers handling VM requests operate without prior knowledge of future VM arrivals, their specifications, or the termination times of active VMs. Consequently, the online version of DVAMP presents a more valuable yet challenging scenario. The mathematical formulation of the online problem is presented in (3).

Definition 4 (Online DVAMP Problem). The online DVAMP problem is characterized by:

- 1) Objective: Minimize the total wait time WT , as defined in the offline version (2a).
- 2) Decision-making: Decisions are made sequentially as each new VM request arrives.
- 3) Available Information: At time t , when deciding for v_j , only information about past and current events is known. This includes the data from (3a) to (3e), encompassing resource requirements, arrival times, start times, completed VM durations, and previous deployment decisions.
- 4) Constraints: The problem is subject to the same constraints as the offline version.

Remark 2. This online problem mirrors real-world scenarios, requiring decisions with incomplete future information to balance current resource use and future demands. For each incoming VM request, the algorithm determines the NUMA node(s) and start time, as shown in (3f) and (3g).

Minimize WT as defined in (2a)

For each $j \in [n]$ and $t > at_j$:

Given:

$$r_l^d \quad \forall l \in [j], \forall d \in [D] \quad (3a)$$

$$at_l \quad \forall l \in [j] \quad (3b)$$

$$st_l \quad \forall l \in [j-1] \quad (3c)$$

$$lt_l \quad \forall l \in [j-1] \text{ s.t. } st_l + lt_l \leq t \quad (3d)$$

$$x_{k,i,l} \quad \forall k \in [m], \forall i \in \{0,1\}, \forall l \in [j-1] \quad (3e)$$

Output:

$$x_{k,i,j} \quad \forall k \in [m], \forall i \in \{0,1\} \quad (3f)$$

$$st_j \quad (3g)$$

Constraints: (2c) – (2k)

III. PERFORMANCE OF ONLINE ALGORITHM

This section examines the performance bounds of greedy online algorithms for the DVAMP problem. In this context, ‘greedy’ implies immediate deployment of a VM if NUMA

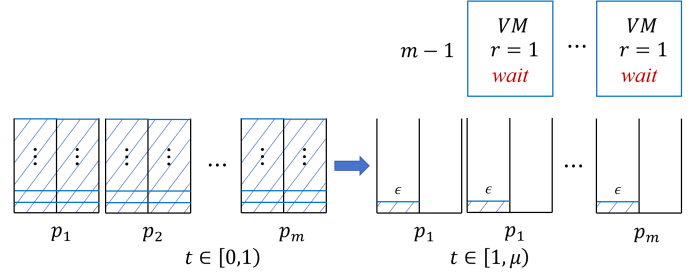


Fig. 2. VM scheduling illustration for a greedy online algorithm. The algorithm initially deploys all VMs arriving at $t = 0$, leading to $m - 1$ VMs waiting when new VMs arrive at $t = 1$.

node(s) are available, as online algorithms cannot determine if delaying deployment would be more beneficial. Let OPT denote the WT value for VM request set V achieved by an optimal offline algorithm, and ON represent the WT value for V of any greedy online algorithm. Given the possibility of $OPT = 0$ and $ON > 0$, the worst-case absolute optimality gap, defined as $\max_V \{ON - OPT\}$, is a more appropriate metric for performance analysis than the competitive ratio.

Several key variables are crucial in this analysis. First, $\mu = \max_j \{lt_j\} / \min_j \{lt_j\}$ represents the ratio of the maximum VM lifetime to the minimum. For simplicity, the minimum VM lifetime is assumed to be 1 and the maximum is μ . Second, $TR = \sum_{d=1}^D \sum_{j=1}^n r_j^d * lt_j$ denotes the total time-resource of VM requests, serving as a descriptive metric for VM request characteristics. The worst-case optimality gap increases as a function of TR , making the ratio of $\max_V \{ON - OPT\}$ to TR particularly significant. The analysis focuses on this ratio, which can be expressed as (4). Both variables are commonly employed in dynamic bin packing problems [13], [14].

$$\max_V \left\{ \frac{ON - OPT}{TR} \right\} \quad (4)$$

The analysis assumes $R_d = 0.5$, implying a maximum resource capacity of 1 for all PMs. Additionally, the resource dimension D is set to 1, which can be achieved when VM resource requirements are identical across all dimensions.

Let q be a large positive integer. Consider a VM request set comprising the following elements:

- m VMs with $r = \frac{1}{2q}$, $at = 0$, $lt = \mu$;
- $2qm - m$ VMs with $r = \frac{1}{2q}$, $at = 0$, $lt = 1$;
- $m - 1$ VMs with $r = 1$, $at = 1$, $lt = 1$.

(For convenience, we omit the VM indices in this notation.)

Any greedy online algorithm will initially allocate all VMs arriving at $t = 0$ to the cluster, with each NUMA node hosting q VMs where $r = \frac{1}{2q}$. At $t = 1$, $2qm - m$ VMs terminate, leaving m VMs active. Due to the inherent limitations of online algorithms, it is impossible to predict which VMs will terminate at $t = 1$. In the worst-case scenario, the m active VMs occupy each PM, preventing the immediate deployment of the $m - 1$ newly arriving VMs with $r = 1$. These new

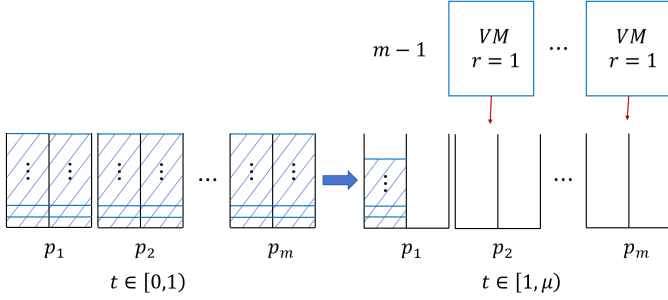


Fig. 3. VM scheduling illustration for the optimal algorithm. By strategically placing VMs active after $t = 1$ on the first NUMA node, the algorithm achieves immediate deployment of new VMs at $t = 1$, minimizing wait time.

VMs must wait until the initially active VMs terminate. Fig. 2 illustrates this process.

The resulting total wait time is $ON = (m-1)(\mu-1)$, with $TR = m(\frac{\mu}{2q}) + \frac{2qm-m}{2q} + (m-1)$. As q approaches infinity:

$$\lim_{q \rightarrow +\infty} \frac{ON}{TR} = \frac{(m-1)}{(2m-1)}(\mu-1) \quad (5)$$

In contrast, an optimal algorithm can place the m VMs that remain active after $t = 1$ on the first NUMA node. This strategy allows immediate deployment of the $m-1$ VMs arriving at $t = 1$ on the remaining $m-1$ PMs, eliminating wait time. Fig. 3 depicts this optimal scheduling process.

In conclusion, for any greedy online algorithm, the worst-case performance relative to the optimal algorithm satisfies (6).

$$\max_V \left\{ \frac{ON - OPT}{TR} \right\} \geq \frac{(m-1)}{(2m-1)}(\mu-1) \quad (6)$$

IV. REINFORCEMENT LEARNING ALGORITHM

A. Reinforcement Learning Framework

To address the complex optimization challenge of the DVAMP problem, we propose a reinforcement learning approach that can adapt to the dynamic nature of VM requests and resource utilization. We formulate the DVAMP problem as a Markov Decision Process (MDP) with partial observability, suitable for reinforcement learning techniques.

Full State (S_{full}): The full state of the system at time t is represented as:

$$s^{full}(t) = (p_1(t), \dots, p_m(t), r_j, at_j, div_j) \quad (7)$$

where $p_k(t)$ represents the full information of all active VMs deployed in p_k at time t . It is defined in the following equation:

$$p_k(t) = \{(st_l, i, r_l) \mid x_{k,i,l} = 1 \text{ and } \delta_l(t) = 1\} \quad (8)$$

Here, $r_l = (r_l^1, \dots, r_l^D)$ is v_l 's resource array, and v_j represents the next VM awaiting deployment. Specifically, the last three elements of (7) (r_j , at_j , and div_j) represent the resource requirements, arrival time, and deployment strategy

of the next VM to be scheduled. If no VM is pending, these three elements are set to null values.

Observable State (S_{obs}): To reduce complexity, the agent observes only the resource utilization of NUMA nodes:

$$s_j^{obs} = (u_1(st_j), \dots, u_m(st_j), r_j, st_j, at_j, div_j) \quad (9)$$

Here, $u_k(st_j) = \{u_{k,i}^d(st_j) \mid \forall i \in \{0, 1\}, \forall d \in [D]\}$ is the resource utilization of p_k at st_j , calculated from active VM information. st_j denotes the deployment time for v_j , which is either its arrival time or the earliest time when at least one NUMA node becomes available for deployment.

Action (A): Action $a_j \in [2m]$ determines the deployment location for v_j . If $div_j = 0$, v_j is deployed in the $\lfloor (a_j+1)/2 \rfloor$ -th server and $[(a_j+1) \bmod 2]$ -th NUMA node; otherwise, it is deployed across both NUMA nodes of the same server.

Reward (R): The reward rew_j is defined as the negative wait time of v_j . This formulation aligns with our objective to minimize total wait time.

$$rew_j = R(s_j^{obs}, a_j) = -(st_j - at_j) \quad (10)$$

Transition (P): We assume discrete time steps, which aligns with the characteristics of real-world datasets such as Huawei-East-1 (to be described in detail in the experimental section). We define two types of transitions:

- Action-based transition: $s^{full}(t) \times a_j \rightarrow s^{full}(t+1)$
- Automatic transition: $s^{full}(t) \rightarrow s^{full}(t+1)$

Action-based transitions occur when a VM is deployed, updating the VM installation status in the full state. The observable state s_j^{obs} is then updated to reflect the changes in resource utilization. Automatic transitions reflect the termination of VMs that have completed their active duration.

Policy (π): The policy $\pi(a_j \mid s_j^{obs})$, denoted as $pr_j^{(a_j)}$, defines the probability of taking action a_j given the observable state s_j^{obs} . For convenience, we define the policy vector as:

$$\pi(s_j^{obs}) = (pr_j^{(1)}, \dots, pr_j^{(2m)}) \quad (11)$$

This reinforcement learning framework provides a structured approach to addressing the DVAMP problem, balancing the trade-off between state observability and computational complexity while adapting to the dynamic nature of VM scheduling in cloud environments.

B. Symmetries of DVAMP Problem

The DVAMP problem exhibits inherent symmetries stemming from the equivalent status of PMs. Under arbitrary permutations of PM states, the policy should undergo a corresponding permutation, reflecting the interchangeable nature of PMs in the system. Specifically, when we apply any permutation to the PM states, the policy vector should be rearranged in the same manner, with the probabilities associated with each PM (represented by pairs of elements due to the two NUMA nodes per PM) following the same permutation.

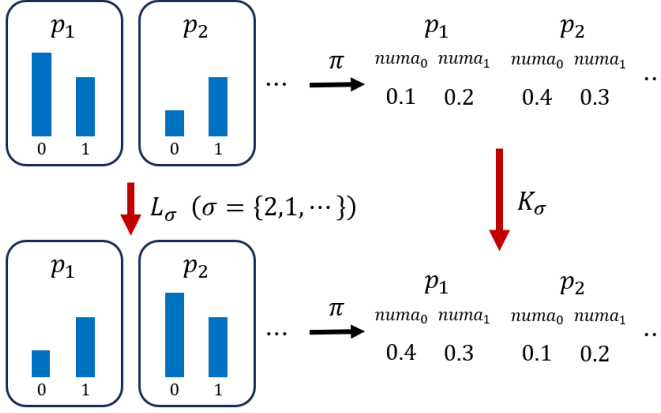


Fig. 4. Illustration of a symmetry-preserving policy in DVAMP. The upper row depicts the original resource utilization of NUMA nodes in p_1 and p_2 , with corresponding policy probabilities. The lower row shows the scenario after swapping the states of p_1 and p_2 . A symmetry-preserving policy ensures that the output probabilities for p_1 and p_2 are correspondingly swapped, maintaining consistency under PM permutations.

To formally describe these symmetries, we employ the mathematical concept of transformation groups. A transformation on s_j^{obs} can be represented as an element of the permutation group of elements $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_m$. We define the transformation group as S'_m , which is isomorphic to the m -element permutation group S_m . For any $\sigma = (\alpha_1, \dots, \alpha_m) \in S_m$, where α_i represents the element that i is mapped to, the corresponding element $L_\sigma \in S'_m$ is defined as:

$$L_\sigma[s_j^{\text{obs}}] = (\mathbf{u}_{\alpha_1}(st_j), \dots, \mathbf{u}_{\alpha_m}(st_j), \mathbf{r}_j, st_j, at_j, div_j) \quad (12)$$

Similarly, the corresponding transformation on π , denoted as K_σ , is defined such that $\forall k \in [m]$:

$$\begin{aligned} K_\sigma[\pi(s_j^{\text{obs}})]_{2k-1} &= pr_j^{(2\alpha_k-1)} \\ K_\sigma[\pi(s_j^{\text{obs}})]_{2k} &= pr_j^{(2\alpha_k)} \end{aligned} \quad (13)$$

where the subscripts $2k-1, 2k$ denote the element indices.

We define a symmetry-preserving policy as one that respects the inherent symmetry of the DVAMP problem. Formally, a policy $\pi(\cdot)$ is considered symmetry-preserving if it satisfies:

$$K_\sigma[\pi(s_j^{\text{obs}})] = \pi(L_\sigma[s_j^{\text{obs}}]) \quad (14)$$

Fig. 4 illustrates this concept, demonstrating how a symmetry-preserving policy maintains consistency under various permutations of PM states.

C. Symmetry Network Architecture

Conventional deep reinforcement learning (DRL) typically employs convolutional neural networks (CNNs) or multilayer perceptrons (MLPs) to approximate Q-functions or policies, but these classical architectures fail to exploit the inherent symmetry in the DVAMP problem. We propose a novel network architecture that inherently satisfies the symmetry constraints, ensuring that the policy trained via DRL adheres

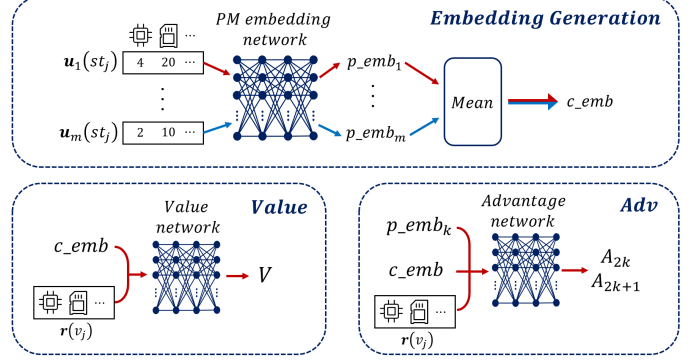


Fig. 5. Symmetry-preserving network architecture for DQN in the DVAMP problem.

to the symmetry condition defined in (14). While we focus on deep Q-learning (DQN) for illustration due to its straightforward implementation, the proposed symmetry network is readily adaptable to other DRL methodologies.

Contemporary DQN implementations often utilize the dueling technique [15], which decomposes the optimal Q-function into two components: the optimal state value function $V(s_j^{\text{obs}})$ and the state-action advantage function $A(a_j, s_j^{\text{obs}})$. This decomposition is expressed as $Q(a_j, s_j^{\text{obs}}) = V(s_j^{\text{obs}}) + A(a_j, s_j^{\text{obs}})$. Post-training, the DQN policy is formulated as:

$$\pi(a_j | s_j^{\text{obs}}) = \begin{cases} 1, & \text{if } a_j = \arg \max_a Q(a, s_j^{\text{obs}}) \\ 0, & \text{otherwise} \end{cases} \quad (15)$$

Given that the total cluster state should be invariant to PM ordering, a symmetry-preserving DQN policy must satisfy:

$$V(s_j^{\text{obs}}) = V(L_\sigma[s_j^{\text{obs}}]), \quad \forall L_\sigma \in S'_m \quad (16)$$

For the advantage function $A(\cdot)$, we define an advantage vector $\mathbf{A}(s_j^{\text{obs}}) = (A(1, s_j^{\text{obs}}), \dots, A(2m, s_j^{\text{obs}}))$ that must satisfy:

$$K_\sigma[\mathbf{A}(s_j^{\text{obs}})] = \mathbf{A}(L_\sigma[s_j^{\text{obs}}]), \quad \forall L_\sigma \in S'_m \quad (17)$$

This implies that $A(2k-1, s_j^{\text{obs}})$ and $A(2k, s_j^{\text{obs}})$ should depend only on p_k and inputs that are invariant to PM ordering.

To achieve this symmetry-preserving architecture, we construct three key modules, as illustrated in Fig. 5.

Embedding Generation Module: This module constructs PM embeddings for each PM and a cluster embedding that is independent of PM ordering. It employs a shared PM Embedding Network across all PMs to generate individual PM embeddings, followed by computing their mean to obtain the cluster embedding. This mean aggregation technique eliminates PM ordering dependencies.

Value Module: In this module, we concatenate the cluster embedding with the resource requirements $\mathbf{r}_j$ and process this through a value network to derive the state value.

Advantage Module: This module employs a common advantage network that takes as input the k -th server embedding,

the cluster embedding, and the resource requirements r_j , outputting $A(2k-1, s_j^{\text{obs}})$ and $A(2k, s_j^{\text{obs}})$.

By combining (15), (16), and (17), we can conclude that our DQN policy satisfies the required symmetry (14). Consequently, our DQN method effectively leverages the symmetry inherent in the DVAMP problem, which is anticipated to yield superior performance, as demonstrated in Section V.

V. PERFORMANCE EVALUATION

A. Experiment Settings

a) *Dataset, Environment*: Experiments use the Huawei-East-1 dataset [12], containing 125,430 VM creation and 116,313 deletion requests over 30 days. As the DVAMP problem considers all requests to have active length time, only requests with deletion data are considered. The active length time is obtained by subtracting the deletion request time from the arrival time. Each request includes two resource requirements: CPU core and memory, with 15 types ranging from 1u1g to 64u128g.

For the cluster, similar to [12], PM number $m = 5$, the resource dimension $D = 2$ which are cpu cores and memory, $R_1 = 40$, $R_2 = 90$, and $div_j = 1$ if $R_2 \geq 10$. For each episode, the environment (consisting of requests and PMs) is reset initially. Upon reset, cluster resource utilization is set to zero. The training set randomly selects initial request indices from the range 0 to 50,000. The validation and test sets randomly select from ranges 50,000 to 70,000 and 70,000 to 110,000, respectively. Both validation and test sets remain fixed throughout the experiments.

b) *Compared Methods*: The proposed method (**SPANE-DQN**) is compared with several heuristic methods: First Fit, Balance Fit, DQN using MLP for approximate Q value (**MLP-DQN**), and MLP with data augmentation (**MLP-DQN-Aug**). The comparison with [12] was not conducted due to differences in the considered objective and the lack of specification of its reward function, which is a key innovation point. Additionally, while [16] proposed a method to exploit arbitrary symmetry of MDP by limiting the linear space of the weight matrix from MLP, its implementation proves challenging due to the lack of support from common modules such as PyTorch for its training procedure.

SPANE-DQN: This method employs DQN with techniques such as dueling [15], double DQN [17], and n -step as the DRL method. It use the SPANE to approximate the Q function. The general training process involves:

- 1) Collecting data from 100 episodes without training; the data are in the form $(s_j^{\text{obs}}, a_j, rew_j, s_{(j+n)}^{\text{obj}})$ and are stored in replay memory, where rew_j is the n -step reward.
- 2) For each epoch, collecting data from one episode, then undergoing a learning procedure from the data in replay memory randomly.
- 3) After a certain interval of epochs, performing validation and obtaining the validation loss. The model with the least validation loss is selected.

Balance Fit: For VMs with $div_j = 1$, this method selects the first PM capable of deploying the VM. For VMs with $div_j = 0$, it first considers only PMs with at least one available NUMA node. Then, it chooses the PM with the largest NUMA nodes resource utilization difference, calculated by (18):

$$\sum_{d=1}^D \frac{|u_{k,0}^d - u_{k,1}^d|}{R_d} \quad (18)$$

It then selects the less utilized NUMA node for VM deployment, aiming to balance the utilization load.

MLP-DQN: This method uses the same DRL method as Sym-DQN but employs MLP to approximate state value and state-action advantages.

MLP-DQN-Aug: In this method, after storing the original data, transformations are applied to enable the MLP to learn the symmetry of the original problem. Specifically, several random $L_\sigma \in S'_m$ are chosen, and the replay memory stores $(L_\sigma[s_j^{\text{obs}}], K'_\sigma[a_j], rew_j, L_\sigma[s_{(j+n)}^{\text{obj}}])$, where K'_σ is defined by (19) and (20):

$$K'_\sigma[a_j] = 2k - i \quad (19)$$

$$k = \sigma(\lfloor (a_j + 1)/2 \rfloor), i = a_j \bmod 2 \quad (20)$$

Here, $\lfloor a_j/2 \rfloor$ and i represent the corresponding server index and the NUMA node index, respectively. The transformation changes the server index from $\lfloor a_j/2 \rfloor$ to $\sigma(\lfloor a_j/2 \rfloor)$, retaining the NUMA index and the reward. The neural network structure for MLP-DQN-Aug is identical to that of MLP-DQN.

c) *Hyperparameters*: Random grid search is employed to determine suitable hyperparameters for the DRL. The hyperparameters and environment parameters are shown in Table II. Best hyperparameters are selected based on the least validation loss. For SPANE-DQN, all three networks have only one hidden layer; the PM embedding network and value network hidden layer have a width of 8, while the advantage network has a width of 16. For both MLP-DQN and MLP-DQN-Aug, the network has two hidden layers with a width of 32. For MLP-DQN-Aug, after storing the original data, 23 random transformations are applied, and the transformed data are stored, resulting in each data point being augmented to 24 versions. To ensure a fair comparison, the training dataset size is kept the same for all DRL methods; hence, MLP-DQN-Aug collects data every 24 epochs.

B. Wait Time Analysis

This section presents a comparative analysis of total wait times across heuristic methods, SPANE-DQN, MLP-DQN, and MLP-DQN-Aug. For each DQN method, 11 experiments were conducted, maintaining consistent parameters and validation sets while varying initialization and random training sets. Figure 6 illustrates the results.

Figure 6 reveals two notable outliers in the SPANE-DQN results. Consequently, a trimmed mean was calculated for each DQN method, excluding the top and bottom two results. We

TABLE II
HYPERPARAMETERS AND ENVIRONMENT PARAMETERS

Parameter	Value
D, R_1, R_2	2, 40, 90
d_{div}, c_{div}	2, 10
Activation function	ReLU
Optimizer	Adam
Epsilon-greedy probability	Linear decay from 0.6 to 0.0
N step	50
L_2 weight decay	10^{-8}
Learning rate	0.01
Epoch	5000
Batch Size	1024
Valid interval	250
Valid data size	150
Test data size	1000

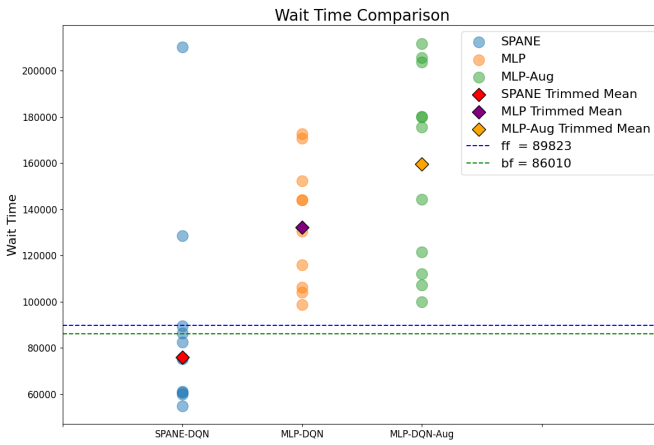


Fig. 6. Wait time comparison across different methods. Circular markers represent mean wait times from 1000 test data points per experiment. The green and blue dashed lines indicate wait times for balance fit and first fit, respectively. Diamond markers denote the trimmed mean for each method.

also calculate the mean of the three experiments with the lowest validation loss for practical model selection. Table III presents these results, demonstrating that SPANE-DQN outperforms all other methods. Specifically, SPANE-DQN achieves wait times that are only 54% of those obtained with MLP-DQN, indicating significantly higher efficiency. Additionally, selecting models with the lowest validation loss consistently yields better results.

TABLE III
TOTAL WAIT TIME FOR DIFFERENT METHODS

Method	Trimmed Mean	Mean of Top 3
SPANE-DQN	75,803	63,697
MLP-DQN	132,144	117,552
MLP-DQN-AUG	159,568	133,852
First Fit	89,823	89,823
Balance Fit	86,010	86,010

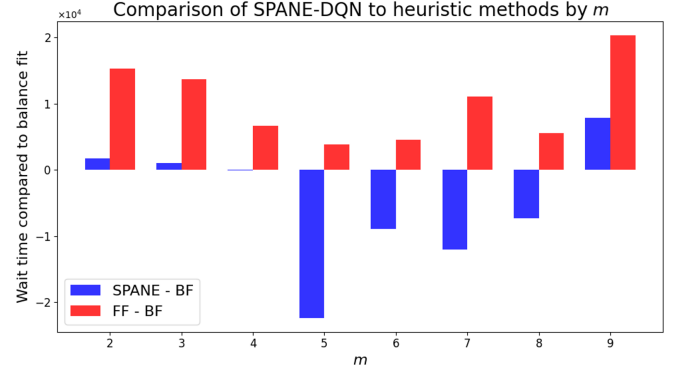


Fig. 7. Comparison of SPANE-DQN to heuristic methods for different numbers of PMs (m). The graph shows the difference in wait time between SPANE-DQN and Balance Fit (blue bars), and between First Fit and Balance Fit (red bars). Negative values indicate better performance than Balance Fit.

The results indicate that both MLP-DQN and MLP-DQN-Aug perform significantly worse than the heuristic methods. This suggests that exploiting symmetry in an intelligent manner, as done in SPANE-DQN, can yield superior results. The poor performance of MLP-DQN-Aug is particularly noteworthy. While data augmentation may enable the DQN to learn the symmetry of the DVAMP problem, it potentially introduces high correlation within the replay memory data, which could adversely affect performance.

C. Flexibility of SPANE

The SPANE neural network architecture demonstrates remarkable flexibility, as it is independent of the PM cluster size. This characteristic allows the SPANE-DQN model to schedule VM requests without retraining when the number of PMs changes. To evaluate this flexibility, we conducted experiments using the top three SPANE-DQN models trained with $m = 5$ PMs. We then compared the total wait time of these models to that of the best heuristic method, Balance Fit, across varying numbers of PMs.

Figure 7 illustrates the comparative performance of SPANE-DQN and First Fit against Balance Fit for various PM counts (m). The y-axis represents the difference in wait time, with negative values indicating superior performance compared to Balance Fit.

The results reveal important insights about SPANE-DQN's performance and flexibility. SPANE-DQN achieves its best performance at $m = 5$, aligning with the training configuration and underscoring the effectiveness of the learning process. More importantly, the graph demonstrates SPANE-DQN's flexibility across different cluster sizes. For $m = 6, 7$, and 8 , SPANE-DQN consistently outperforms Balance Fit, despite being trained on a different PM count. This adaptability indicates that the learned scheduling strategies can generalize to larger cluster sizes without retraining.

At $m = 4$, SPANE-DQN's performance is slightly inferior to Balance Fit, though the difference is marginal. This suggests that the model maintains relatively competitive performance even when applied to cluster sizes smaller than its training configuration. The performance variation across different m values hints at potential differences in optimal scheduling strategies for varying cluster sizes. For larger values of m (5 and above), SPANE-DQN generally outperforms Balance Fit, which may indicate that the model's learned strategies are particularly effective for these cluster sizes.

These observations raise interesting questions about the relationship between cluster size and optimal scheduling strategies. It's possible that as cluster size increases, there might be some convergence in effective scheduling approaches, which could potentially benefit methods like SPANE-DQN that are designed to capture complex patterns. For smaller clusters, the performance differences suggest that optimal strategies might vary more significantly, presenting an opportunity for further investigation and potential refinement of the SPANE-DQN model for these scenarios.

The consistent outperformance of both SPANE-DQN and Balance Fit over First Fit across all m values emphasizes the value of sophisticated scheduling methods in VM allocation tasks, highlighting the limitations of simpler heuristics in complex scenarios.

These findings underscore the flexibility and effectiveness of the SPANE-DQN approach in VM scheduling across various cluster sizes. The ability to maintain and often improve performance without retraining represents a significant advantage in terms of computational efficiency and adaptability to changing infrastructure requirements.

VI. RELATED WORK

A. Dynamic VM Scheduling Problem

The dynamic VM Scheduling (DVMS) Problem is a specific application of the dynamic bin packing (DBP) in the cloud computing environment. There has been a large amount of prior work focusing on DVMS with different objective functions. Some works focus on minimizing the number of hosts [2], [3], which is the same as the DBP. A large amount of research works have been done to analyze the competitive ratios of various algorithms for DBP. Coffman et al. [13] proved that First Fit algorithm has competitive ratios between 2.75 and 2.897. Ivkovic et al. [14] generalized the problem by considering that packed items can be moved to different bins. Some works focus on minimizing the usage time of the hosts [4], [5], [18]. Tang et al. [4] proposed an algorithm to minimize the host run time for DVMS. Liu et al. [5] predicted the duration for each item and develop an online algorithm with consistency and robustness. Some works consider different server architecture and maximize the number

of created VMs during scheduling [9], [12]. Sheng et al. [9] considered the multi-NUMA architecture and formulated it into a reinforcement learning framework. Then, an efficient RL-based algorithm is proposed to maximize the number of scheduled VMs. In contrast with these work, our work treats the VM waiting time as objective and analyze the competitive ratio under multi-NUMA scenario.

B. Reinforcement Learning for Scheduling

Cloud computing environments are becoming increasingly complex and volatile due to unpredictable user behaviors, heterogeneous servers, and a high volume of requests. To effectively schedule resource in the cloud, deep reinforcement learning has demonstrated considerable advantages in many complex scheduling scenarios [10], [11], [19]–[22]. Wang et al. [11] optimized resource scheduling based on QoS feature learning and improved Q-learning algorithm. Chen et al. [23] proposed a prediction-based Q-learning algorithm to adaptively allocate resources for dynamic cloud workloads. Siddesha et al. [19] handled different sizes of tasks and varied number of VMs with deep RL. However, the state space and the action space for these problems are extremely huge, which leads to poor convergence and a large overhead. Most works also assume that the VM last time is known in advance. In this work, we focus on the dynamic scheduling problem (the VM last time is unknown until it is explicitly stopped by the user) and consider the multi-NUMA server architecture, which makes the state and action space more complex.

VII. CONCLUSION

We introduces a novel variation of the dynamic vector bin-packing problem, referred to as DVAMP, within the context of multi-NUMA systems. We formally define the problem, providing MILP formulations for both offline and online versions, and derive a performance bound for the greedy online algorithm. To address the challenges posed by DVAMP, we propose SPANE, a symmetry-preserving deep reinforcement learning framework that ensures allocation results are invariant to permutations of physical machine states, thereby improving both learning efficiency and solution quality. Extensive experiments on real-world cloud datasets demonstrate that our proposed RL algorithm significantly outperforms existing baselines, achieving notable reductions in VM wait time.

ACKNOWLEDGMENT

This work was supported by the National Key R&D Program of China [2024YFF0617700], the National Natural Science Foundation of China [U23A20309, 62172276, 62372296, 62272302], Shanghai Municipal Science and Technology Major Project [2021SHZDZX0102], and the Huawei Cloud [TC20230526044].

REFERENCES

- [1] J. Shi, J. Luo, F. Dong, J. Jin, and J. Shen, "Fast multi-resource allocation with patterns in large scale cloud data center," *J. Comput. Sci.*, vol. 26, pp. 389–401, 2018.
- [2] A. Murhekar, D. Arbour, T. Mai, and A. B. Rao, "Dynamic vector bin packing for online resource allocation in the cloud," in *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. ACM, 2023, pp. 307–310.
- [3] J. W.-T. Chan, P. W. Wong, and F. C. Yung, "On dynamic bin packing: An improved lower bound and resource augmentation analysis," *Algorithmica*, vol. 53, no. 2, pp. 172–206, 2009.
- [4] X. Tang, Y. Li, R. Ren, and W. Cai, "On first fit bin packing for online cloud server allocation," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2016, pp. 323–332.
- [5] M. Liu and X. Tang, "Dynamic bin packing with predictions," in *Abstract Proceedings of the 2023 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, ser. SIGMETRICS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 57–58.
- [6] C. Lameter, "Numa (non-uniform memory access): An overview: Numa becomes more common because memory controllers get close to execution units on microprocessors," *Queue*, vol. 11, no. 7, p. 40–51, Jul. 2013.
- [7] "Aws ground station agent user guide: Cpu assignment example," accessed: 2025-01-11. [Online]. Available: <https://docs.aws.amazon.com/ground-station/latest/gs-agent-ug/agent-instance-selection.html>
- [8] J. Rao, K. Wang, X. Zhou, and C. Xu, "Optimizing virtual machine scheduling in NUMA multicore systems," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2013, pp. 306–317.
- [9] J. Sheng, Y. Hu, W. Zhou, L. Zhu, B. Jin, J. Wang, and X. Wang, "Learning to schedule multi-NUMA virtual machines via reinforcement learning," *Pattern Recognition*, vol. 121, p. 108254, 2022.
- [10] G. Zhou, W. Tian, R. Buyya, R. Xue, and L. Song, "Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions," *Artificial Intelligence Review*, vol. 57, no. 5, p. 124, 2024.
- [11] B. Wang, F. Liu, and W. Lin, "Energy-efficient vm scheduling based on deep reinforcement learning," *Future Generation Computer Systems*, vol. 125, pp. 616–628, 2021.
- [12] J. Sheng, S. Cai, H. Cui, W. Li, Y. Hua, B. Jin, W. Zhou, Y. Hu, L. Zhu, Q. Peng, H. Zha, and X. Wang, "VMAgent: A Practical Virtual Machine Scheduling Platform," in *IJCAI*, vol. 6, Jul. 2022, pp. 5944–5947.
- [13] E. G. Coffman, Jr, M. R. Garey, and D. S. Johnson, "Dynamic bin packing," *SIAM Journal on Computing*, vol. 12, no. 2, pp. 227–258, 1983.
- [14] Z. Ivkovic and E. L. Lloyd, "Fully dynamic algorithms for bin packing: Being (mostly) myopic helps," *SIAM Journal on Computing*, vol. 28, no. 2, pp. 574–611, 1998.
- [15] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *International conference on machine learning*. PMLR, 2016, pp. 1995–2003.
- [16] E. Van der Pol, D. Worrall, H. van Hoof, F. Oliehoek, and M. Welling, "Mdp homomorphic networks: Group symmetries in reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 4199–4210, 2020.
- [17] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.
- [18] D. Kong, Z. Zhang, Y. Shi, and X. Gao, "An approximation for job scheduling on cloud with synchronization and slowdown constraints," in *IEEE Conference on Computer Communications (INFOCOM)*, 2023, pp. 1–10.
- [19] K. Siddesha, G. Jayaramaiah, and C. Singh, "A novel deep reinforcement learning scheme for task scheduling in cloud computing," *Cluster computing*, vol. 25, no. 6, pp. 4171–4188, 2022.
- [20] Z. Peng, D. Cui, J. Zuo, Q. Li, B. Xu, and W. Lin, "Random task scheduling scheme based on reinforcement learning in cloud computing," *Cluster computing*, vol. 18, pp. 1595–1607, 2015.
- [21] M. Cheng, J. Li, and S. Nazarian, "Drl-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers," in *2018 23rd Asia and South Pacific design automation conference (ASP-DAC)*. IEEE, 2018, pp. 129–134.
- [22] F. Cheng, Y. Huang, B. Tanpure, P. Sawalani, L. Cheng, and C. Liu, "Cost-aware job scheduling for cloud instances using deep reinforcement learning," *Cluster Computing*, pp. 1–13, 2022.
- [23] X. Chen, F. Zhu, Z. Chen, G. Min, X. Zheng, and C. Rong, "Resource allocation for cloud-based software services using prediction-enabled feedback control with reinforcement learning," *IEEE Transactions on Cloud Computing*, vol. 10, no. 2, pp. 1117–1129, 2020.